

Inhomogeneity Detection using Artificial Neural Networks Solutions to the Inverse Cauchy Problem

CE57003 MTECH PROJECT-II

Report submitted in
Partial fulfillment for the award of the Degree of

Master of Technology in Civil Engineering
With specialization in
Structural Engineering

by
Pranav Shukla
17CE31004

under the supervision of
Dr. Biswanath Banerjee



Department of Civil Engineering
Indian Institute of Technology
Kharagpur

April 2022

Declaration

I hereby certify that

- (a) The work contained in this report has been done by me under the guidance of my supervisor.
- (b) The work has not been submitted to any other Institute for any degree or diploma.
- (c) I have followed the norms and guidelines given in the Ethical Code of Conduct of the Institute.
- (d) Whenever I have used materials (data, theoretical analysis, figures and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

Date: 25th November, 2021
Place: IIT Kharagpur

Pranav Shukla
17CE31004

DEPARTMENT OF CIVIL ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY,
KHARAGPUR
KHARAGPUR-721302, INDIA



CERTIFICATE

This is to certify that the Thesis entitled “Inverse Inhomogeneity Detection using Iterative Artificial Neural Networks” is a bona fide record of authentic work carried out by Pranav Shukla under my supervision and guidance for the partial fulfillment of requirements for the award of degree of Master of Technology in Structural Engineering in the department of civil engineering at the Indian Institute of Technology Kharagpur.

Date: 25th November, 2021
Place: IIT Kharagpur

Dr. Biswanath Banerjee
Associate Professor
Department of Civil Engineering
Indian Institute of Technology Kharagpur

Acknowledgement

I would first and foremost like to thank my supervising Professor, **Dr. Biswanath Banerjee** who granted me the opportunity to take up this project and who guided and steered me in the right direction throughout the project with his invaluable expertise. Your insightful feedback and genuine interest in the project always ended up amplifying my own interest and drove me to sharpen my thinking and accumulate as much knowledge as possible to bring the project to completion.

I would also like to thank my parents, without whose grace i wouldn't have gotten so much as near this opportunity far alone taking it to completion. I also thank them for bearing with me and always being there for me unconditionally.

Date: 25th November, 2021

.....
Signature of Student

Abstract

The following report proposes and validates a methodology for estimation of extent and location material deformations in structures using partial boundary and partial domain information availability using alternating minimization of Neural Network based solutions to inverse Cauchy Problems for time-independent two dimensional systems. The report demonstrates the convergence of 2 dimensional partial-information ill-posed boundary value problems by construction of a neural network of variable width and multiple hidden layers which offers highly non-linear kernels for mapping. The report then proceeds to propose and validate an iterative procedure for estimation of optimal material constants for the domain for detection of non-homogeneity and deformations, through alternating minimization of a novel cost functional.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Formulation	3
1.3	Objective and Scope of Work	4
2	Literature Review	5
3	Mathematical Model	7
3.1	Network model	8
3.1.1	Activation function	9
3.1.2	Cost functions	9
3.2	Computing exact solution of the problem	10
3.3	Optimization of Material Matrix	10
3.3.1	Convergence Analysis of the Optimizer	11
4	Experimental Validation	12
4.1	<i>Finite Element</i> Solution- using Complete boundary information .	13
4.2	Solution- using <i>Partial Boundary Condition</i>	14
4.3	Threshold Filters for low information problems	16
4.4	Results	18
5	Conclusion and Scope for future work	19

List of Figures

3.1	Architecture for Ill-posed problems	8
3.2	General structure of a neural net.	9
3.3	Architecture for Well-posed problems	11
4.1	Problem Domain	12
4.2	True Material Matrix	13
4.3	Initial Guess(ITR-0) for the Material Matrix	13
4.4	True Material Matrix	15
4.5	Initial Guess(ITR-0) for the Material Matrix	15

Chapter 1

Introduction

1.1 Motivation

Prediction of material constants and deformations from the inverse Cauchy problem has been an important obstacle to solve in various domains of applications in the past few years. What solving this problem would effectively mean is, we would be able to model systems, only with partial boundary data available to us. This would be a great leap forward in terms of non-destructive testing. For instance, we would be able to detect cracks in pipes which only have parts of it exposed for data collection. Or we would be able to design a model to predict temperatures inside a combustion chamber, using the material properties and the boundary data available to us on the outer surface. Solving this problem in a noise resistant fashion would have impacts in the domain of neuroscience as well. Since we can only extract electrical activity data from parts of the brain non intrusively, so by modelling the brain as a multi dimensional system, we could use this architecture to predict approximations for electric field and neural activity using data available to us only on parts of the boundary, which in turn, would create significant scope for development in that discipline.

1.2 Formulation

A Cauchy problem in mathematics asks for the solution of a partial differential equation that satisfies certain conditions that are given on a hyper-surface in the domain. The inverse Cauchy problem has both the Neumann and Dirichlet boundary conditions on part of the boundary and no information on the rest of the boundary. In such a scenario, the problem becomes ill-posed and becomes very difficult to be explored by standard analytical techniques. For the time independent case, the problem can be formulated as,

$$\begin{aligned}\nabla(m.\nabla u(x)) &= q_0 \quad x \text{ in } \Omega \\ u(x) &= f \quad x \text{ in } \Gamma\end{aligned}$$

$$\frac{\partial u(x)}{\partial n} = g \quad x \text{ in } \Gamma$$

Where $\Omega \subset \mathbb{R}^d$ is a domain with continuous boundary $\partial\Omega$, with d being the spatial dimension, and $\Gamma \subset \partial\Omega$ is a part but not all of $\partial\Omega$, n is the outer unit normal with respect to $\partial\Omega$, $\mathcal{L}$ is a linear operator and f, g are functions/functional in x .

Denoting $x = I(y)$ as an inverse problem x given data y , we simultaneously have to solve two inverse problems to reach at the optimal solution. These are given as:

$$\begin{aligned} u &= I1(f, g, q_0, m) \\ m &= I2(u) \end{aligned}$$

1.3 Objective and Scope of Work

The objective of the report is to produce near optimal material constant values at each point across the domain. This is done by alternating minimization of a λ function of three variables, $m, \sigma, \nabla w$. Where σ and ∇w are exact solutions to the exact and modified inverse Cauchy problems, given a constant m , and m is updated through the extremization of λ .

The scope of this report is limited to evaluating second order differential equations across the domain, given the Dirichlet and Neumann boundary conditions on partial boundary of the domain, with the solution field values of the solution known at arbitrary points inside the domain. We consider a discrete change in the material constant value for a region inside the domain. The report is limited to evaluation of a two dimensional domain but without loss of generalisation.

Chapter 2

Literature Review

The concept of ill-posedness, that is negating the properties of a well-posed problem, comes from the pioneering work of Hadamard who defined existence, stability, and uniqueness of a problem in the context of partial differential equations. The critical feature when looking for a numerical method as a solution to the inverse Cauchy problem is to treat the ill-posedness. Even though there exists a unique solution to the problem, this would mean that due to the inherent ill-posedness of the problem, even small disturbances in the input data might lead to much more amplified errors in the result as referred to here^[6]. And since the realistic boundary measurements are not one unbridled by noise, it then becomes a priority to find accurate as well as stable numerical algorithms to solve the resulting ill-posed algebraic problem.

Regularization methods have been a popular and effective way to treat this inherent ill-posedness of the inverse problem. These methods seem to tackle the stability issue of the problem and aim to convert the problem to an approximate well-posed formulation using some form of iterative transformations. The most popular one by far is the Tikhonov's regularization method^[8], which does this transformation by minimizing the L_2 -norm of the solution subject to constraining parameters, using the L-curve concept to find the optimal parameters, which find the best balance between the norm of the solution and maximum permissible distortions to the original equations. Other classical methods for achieving stability over the past few decades that have been looked into include boundary element method^[5], variational method^[7], an energy minimizing approach^[2], the truncated singular value decomposition method^[4], methods using quasi-reversibility^[3], The method of fundamental solutions(MFS)^[13], etcetera.

Deep artificial neural network methods for solving ill-posed PDEs are a more novel way of approximating such problems. These methods have also found to be convergent to the unique solution and have been proved to be highly resistant to input noise. Moreover, these methods do not require continuous iterative recomputations, as in the case of regularization methods to extract field values at different intermediate points. Where these feed-forward deep learning models exceptionally hold an advantage over the classical methods is

their ability to handle higher dimensional data with much more ease. As the spatial dimensions of the input data grow, the computational costs of the ANN do not grow as quickly as those of the traditional methods.

There have been several great examples of novel combinations of ANN with traditional methods to enhance performance well. Like how Mishra^[11], used an ANN framework on top of an existing finite differential method, or the development of physics informed ANN models^[12] to solve partial differential equations. ANNs by nature, are data-based approximation models so the most important problems in using them for scientific purposes is the integration of domain information into them. This problem can be quite complex to approach due to the restrictions in the number of parameters we can really tinker in a network. To tackle this issue, attempts have been made to utilise encoder-decoder structures to use self-learning of domain knowledge in lower-dimensional latent spaces, cost functions alterations have been used to feed in characteristics equations (collocation), using transfer learning mechanisms to utilise networks specifically trained into specific domain sets, etc.

In the given approach, a neural network is used to represent the output field vector and impose the PDE constraints and partial boundary conditions via a collocation type method^[9].

Chapter 3

Mathematical Model

To sequentially solve the given problem, first, two neural network named *Unet* and *Wnet* respectively are generated. The architecture of both of these networks, as discussed below, is identical, with the difference being in the cost functions which are extremized to train both networks. These cost functions are listed in sub section 3.1.2. These networks take dirichlet and neumann data at the partial boundary, along with field values at random pointst (for *Wnet*) and return an estimate field variable as a solution to the inverse problem across the domain. An initial estimate, $\bar{u}$ and $\bar{w}$ are then computed from these two neural networks using a homogeneous value of material matrix m across the domain. This solution is extracted on the complete boundary and is then used to compute a more exact Finite Element Solution less prone to errors. This is done because of the inherent behaviour of these networks to generate more accurate results on the boundary and the errors propagating as we head inside the domain.

Then, based on the alternating minimization formulation in Section 3.3, the material matrix values are updated. Iterations of this scheme are carried out until the material matrix converges to an optimum value. The architecture as such, of the entire model is given by Fig 3.1. Note here that FEM_x represents the finite element solution on the complete boundary for the variable x .

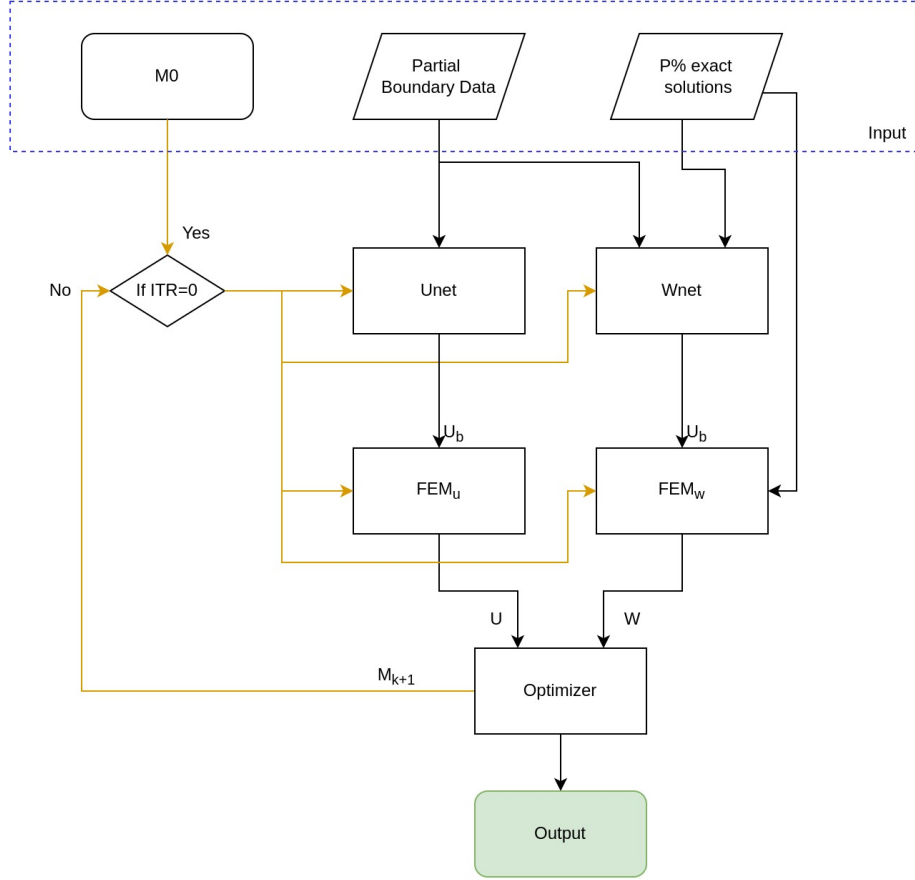


Figure 3.1: Architecture for Ill-posed problems

3.1 Network model

We now consider a deep fully connected feed forward Artificial Neural Network to solve the posed problem. The network consists of $L+1$ layers with L_0 being the input layer and L_L being the output layer and $L-1$ intermediate hidden layers in between. Each layer is activated by a activation function σ which adds non linearity to the network to increase mapping precision. A standard structure of a artificial neural network looks as in Figure 3.2

This is the structure of the standard neural network $Y = F(X)$, which is a mathematical mapping from $\mathbb{R}^X \Rightarrow \mathbb{R}^Y$. Each neuron in a layer corresponds to a z^l value, and an activation function σ . The output of each neuron, for $l > 0$, is computed as follows:

$$z^l = w^l y^{l-1} + b^l$$

$$y^l = \sigma_l(z^l)$$

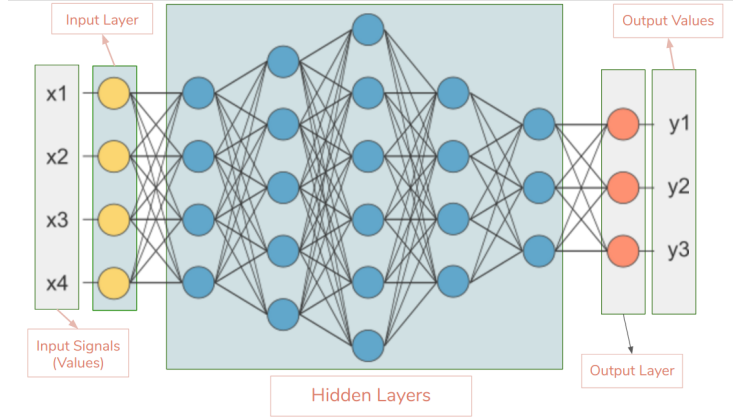


Figure 3.2: General structure of a neural net.

Here, y^l is the output of the neuron in layer l and w^l, b^l are the neuron parameters which have to be trained. The set of all w^l, b^l for all neurons are called the network parameters. The output of the last layer, ie, y^L is the output of the network.

$$NET(X, W, B) = y^L$$

The the initialization of the network parameters, the **backward propagation** of the network and the modified version of gradient descend (ADAM optimiser) on the network parameters is computed using **tensorflow 2.0**^[10] and numerically computed gradients of the cost function.

3.1.1 Activation function

Here, we implement the network using the sigmoid activation function as it keeps the layer values normalised and the second order differential does not converge to zero in most of its Domain. We choose not to use the more computationally inexpensive ReLu activation function because it converges to zero during higher order numerical differentiation. ReLu is feasible for analytically computed back propagation but for numerical analysis, sigmoid is the better choice.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

3.1.2 Cost functions

The cost function penalises the network based upon the weighted summation of the values of the given equation and their convergence to zero. This is very similar to the idea of the weak formulation in Finite element analysis. With the difference being discrete point inputs inclusion of both, the natural and the

essential conditions in the cost function given by:

$$J(\bar{u})_{U_{net}} = k_1 \|\nabla(m \cdot \nabla \bar{u}) - q_0\|_{L_2(\Omega)}^2 + k_2 \|\bar{u} - f\|_{L_2(\Gamma)}^2 + k_3 \left\| \frac{\partial \bar{u}}{\partial n} - g \right\|_{L_2(\Gamma)}^2$$

$$J(\bar{w})_{W_{net}} = J(\bar{w})_{U_{net}} + k_4 \|\bar{w} - w^t\|_{L_2(S(N'))}^2$$

This is a modification on the cost function in the paper by Yixin Li and Xianliang Hu [9], where k_1, k_2 and k_3 and k_4 are weight constants. $w^t \in S(N')$ are true field values sampled at N' random points inside the domain. The constant k_4 is set to be 10^5 order of magnitude higher than the rest so as to emphasize on the penalty of this term.

The goal is such as to minimize the cost function J with respect to nodal parameters of the neural network where $\bar{u}$ is the estimated field output generated from the ANN given by

$$\bar{u} = UNET(X, W, B)$$

$$\bar{w} = WNET(X, W, B)$$

NOTE: from here on forth, we will refer to u, w as output of the neural network, unless specified otherwise.

3.2 Computing exact solution of the problem

To train the networks and to validate their performance, the exact solution of the full boundary finite element problem is computed using the FEniCS^[1] framework and the resultant field values are sampled for the cost function of W_{net} and to check convergence performance of U_{net} .

3.3 Optimization of Material Matrix

We define a multivariate cost function λ for every point in the domain, given as:

$$\lambda(m, \sigma, \nabla w) = \frac{1}{2} \|\sigma - m \cdot \nabla w\|^2$$

$$where, \sigma = m \cdot \nabla u$$

The idea of Alternating Minimization (AM) method is to decouple the data and to repeatedly update each variable by extremizing λ using previous values of the remaining variables. To update σ and w we use their respective neural networks. To update m , we write the partial derivative of λ with respect to m , under the AM assumption that $\sigma, \nabla w, m$ are independent (even if they are not, update is still consistent).

$$\frac{\partial \lambda}{\partial m} = (\sigma - m \cdot \nabla w) \cdot \nabla w = 0(extremization)$$

$$\Rightarrow m = \frac{\sigma}{\nabla w}$$

Using $\sigma = m_k \nabla u$, we can formulate the update function as:

$$m_{k+1} = \frac{m_k \cdot \nabla u_k^T \cdot \nabla w_k}{\nabla w_k^T \cdot \nabla w_k}$$

3.3.1 Convergence Analysis of the Optimizer

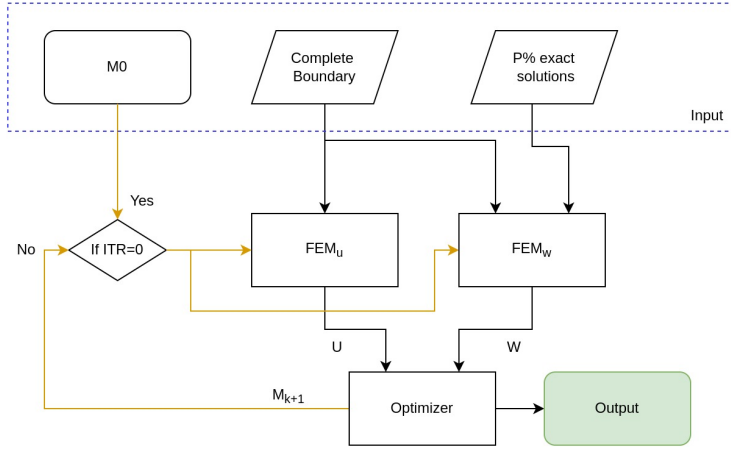


Figure 3.3: Architecture for Well-posed problems

Before applying the optimizing function on the neural network model, we first use a well-posed finite element problem to check the convergence of the alternating minimization formulation. The architecture of said formulation is given by fig 3.3. The convergence of the formulation is also demonstrated in Chapter 4.

Chapter 4

Experimental Validation

The validation of the model is twofold, first we validate the convergence of the material matrix optimization function using two finite element solutions of the model with the complete boundary equations available. We then follow up with testing the Convergence of the solution with the incomplete boundary information using the neural networks. We consider the time independent problem given by the following equations on the two dimensional domain in Fig 4.1:

$$\begin{aligned}\nabla(m.\nabla u(x)) &= -4 \quad x \text{ in } \Omega \\ u(x) &= 1 \quad x \text{ in } \Gamma\end{aligned}$$

Domain Ω and boundary Γ are shown in Fig 4.1, where radius of the domain is 1. Our objective is to determine m and its variation across the domain using our inverse formulation.

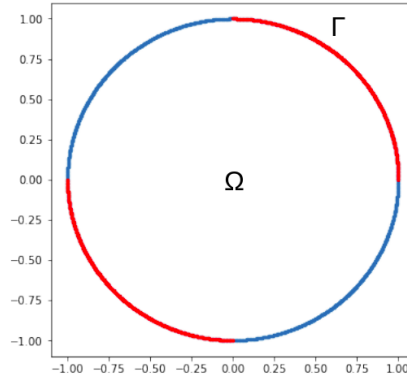


Figure 4.1: Problem Domain

4.1 *Finite Element* Solution- using Complete boundary information

The Convergence of the optimization function give in Section 3.3 is evaluated using two Finite element solutions of the system given by u and w respectively. The solution u is evaluated by applying the given boundary condition on the entire boundary and by using the initial guess for the material matrix. The second solution, w is also evaluated on the same material matrix and complete boundary conditions but with the key difference being that it is forced with p percent of additional points to to be more closer to the exact solution of the system. (using the finite element solution of the true material matrix on the domain). This forcing is done by applying another additional Dirichlet boundary condition containing these p percent extra points and their exact solution values.

Note: p is defined as: $p = 100 * \frac{(No.ofpointsselectedrandomly)}{(totalnumberdiscretisedpointsindomain)}$

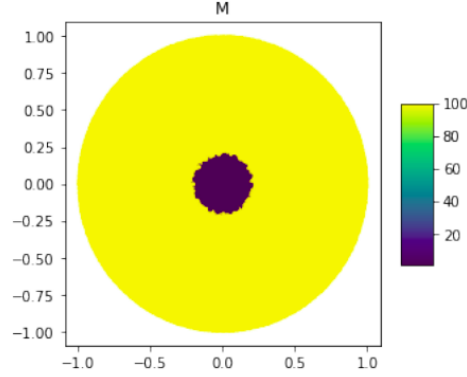


Figure 4.2: True Material Matrix

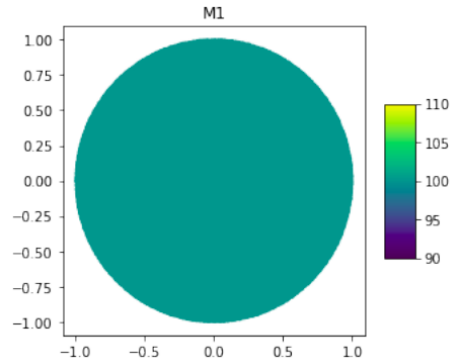
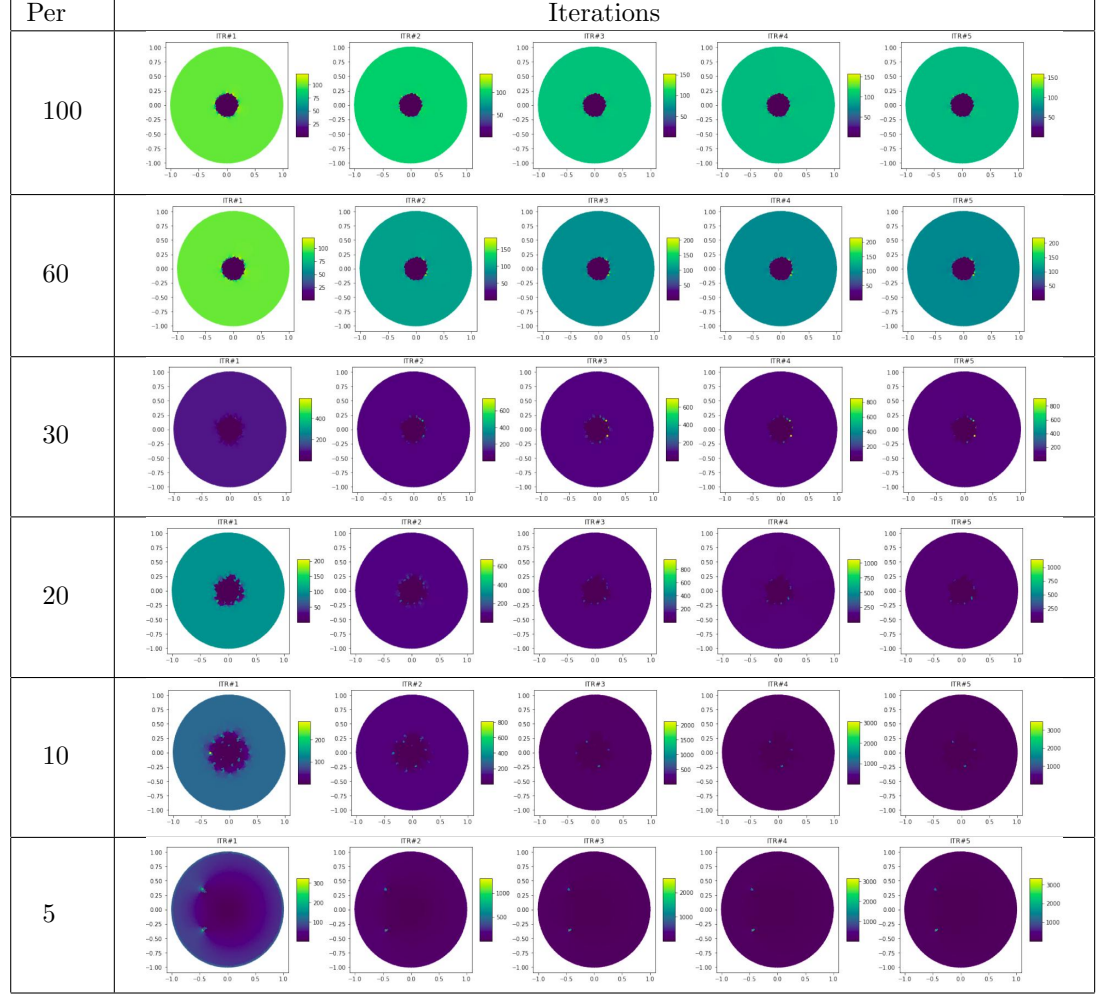


Figure 4.3: Initial Guess(ITR-0) for the Material Matrix

We now evaluate the optimizer for $p = [100, 60, 30, 20, 10, 5]$ over 5 iterations of convergence.



4.2 Solution- using *Partial Boundary Condition*

We now deploy the the neural network model to solve the problem using partial boundary information only. Using the formulated Unet and Wnet in Section 4.1 and 4.2, we use the m update formulation derived in Section 3.3 to update the current material matrix. For this, we initialise the material matrix with a uniform value, $m = 100$ across the entire domain. The first neural network Unet, is supplied with the partial boundary information only. The second neural

network Wnet, is also supplied the additional exact solution points to force it to be more close to the true solution of the domain. The extent of this supply is represented by p .

For the two dimensional example, the structure in the ANN is chosen with layers [2, 120, 40, 20, 12, 10, 5, 1], with w and b initialized with tensorflow's Normal initializer and using it's presets values for the ADAM optimiser ($\beta_1 = 0.9$ and $\beta_2 = 0.999$). The true m matrix used for the analysis, both in ANN and FeNiCS consists of discrete values: [1,100] and their distribution is given by Fig 4.4. The initial guess for every trial is given by Fig 4.5

For the training data, we sample 1200 points from the domain Ω and 400 points from the boundary Γ . We iterate the training over 2000 epoch cycles using 128 mini batches in every iteration of the trial for both the networks. The cost function weights are set as follows: [$k_1 = 10, k_2 = 100000, k_3 = 100, k_4 = 10^6$]. The learning rate is set at 0.0002.

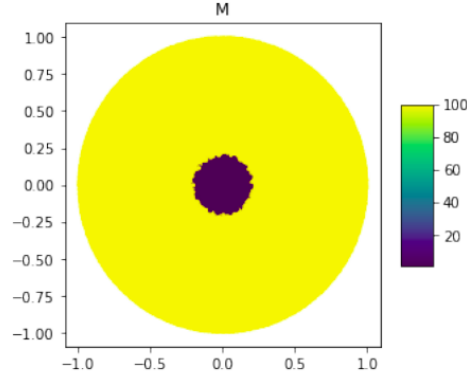


Figure 4.4: True Material Matrix

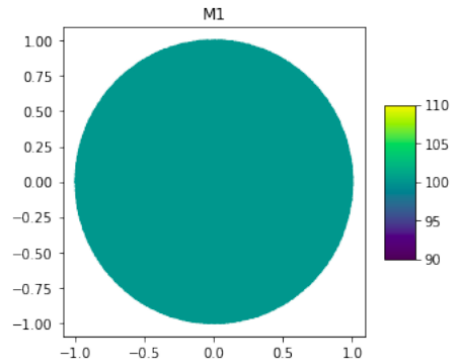
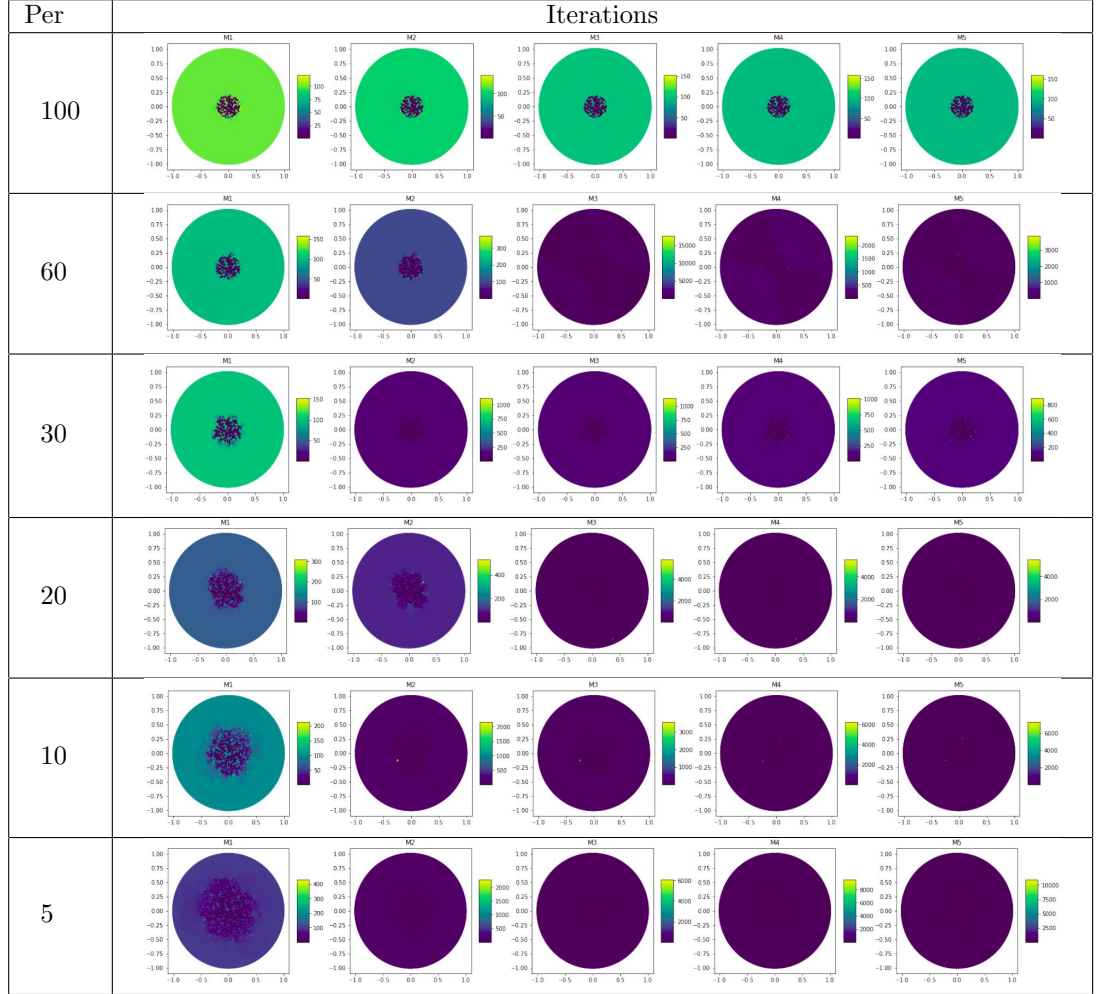


Figure 4.5: Initial Guess(ITR-0) for the Material Matrix

We now evaluate the optimizer for $p = [100, 60, 30, 20, 10, 5]$ over 5 iterations of convergence. The updated material matrix after each iteration of the optimizer is given below.



4.3 Threshold Filters for low information problems

From the results for the complete finite-element and the neural network models, we can notice that when the percentage of available information inside the domain is low, the deformation output by the model contains higher number of erroneous points. A much clearer and objective picture can be drawn if these erroneous points are removed under a threshold parameter on the update

function. The results below consider the following sample threshold filter.

$$M_f = M_s + \chi(M_s)(100 - M_s)$$

where, M_f =filtered solution; M_s =Converged Solution and,

$$\chi(x) = \begin{cases} 1 & x \geq 20 \\ 0 & x < 20 \end{cases}$$

Note that this is an optional physics-informed parameter which should only be selected if we have information about the nature of the deformations. The results below show the effect of applying this filter on the results in section 4.2 assuming a threshold of $M = 20$. We notice significant improvement in the location and extent of deformations based on the proposed filtering.



4.4 Results

From the convergence results in Section 4.2 we can validate the performance of the proposed alternating minimization formulation. The result show that the model given in Fig 3.3 works well in converging to the optimum solution for p percentage values of range (10, 100). Below that, the variations in the optimizer updates start to vary erroneously.

From the results in Section 4.1 we can see that both the neural networks on their own work well to predict the field values and the approximated boundary values they supply work well to converge the material matrix to their true value. We notice approximation errors at the unknown boundary locations but these remain minimal so as to not affect the convergence of the model. For low p value iterations, the results at distinct points jump to large values rendering the rest of the plot unreadable. Upon filtering such limited erroneous points, much more informative results are generated, as seen in Section 4.3.

From the results in Section 4.2, We also arrive at the minimum percentage of points with supplied exact solution required to get a well-performing model. This, for the problem at hand can be reported to be greater than 10 percent. This is an important parameter for consideration in all such problems and gives us a limit of the minimum number of experimental observations necessary to model such systems well.

Chapter 5

Conclusion and Scope for future work

We have investigated an artificial neural network approximation approach to for solving the Inverse Cauchy Problem using partial boundary data. And using this solution we have devised an iterative scheme to find the optimal material matrix. This matrix can be used to detect rapid changes in the material structure and detect deformations/damages.

Through our results, we can establish that this neural network works well in predicting the field variable in the absence of complete boundary information with errors maximising up to 0.0075. We notice this error maximises as we head away from the boundary, that is towards the center of the domain. This error cannot be further reduced upon application of regularisation techniques like L_2 regularisation or dropout regularisation on the network parameters. This is so because the test sets and training sets are not mutually exclusive, so this predominantly is a bias and not a variance problem.

To circumvent this problem, a methodology can be to extract the less error prone points at the boundary and with this complete boundary information of the domain then create a complete finite element solution to the problem. Upon adopting this methodology we notice results for the convergence of the material matrix are much more accurate.

We have also utilised an arbitrary (point based rather than shape defined) material matrix enabling us to extend this approach to deformations of any shape and dimension.

Through the convergence analysis that was carried out, we have also found out a minimal information criteria for the solutions to converge. This is the minimum information about the real system required to predict the locations and extend of the deformity well.

In future works, we can look at more novel structures and extend the functionality of the model to handle higher space dimensionality and handle time-dependent problem

Bibliography

- [1] Martin S. Alnæs et al. “The FEniCS Project Version 1.5”. In: *Archive of Numerical Software* 3.100 (2015). DOI: 10.11588/ans.2015.100.20553.
- [2] Stéphane Andrieux, Thouraya Baranger, and Amel Ben Abda. “Solving Cauchy problems by minimizing an energy-like functional”. In: *Inverse Problems* 22 (Feb. 2006), pp. 115–133. URL: <https://hal.archives-ouvertes.fr/hal-00139569>.
- [3] Laurent Bourgeois. “A mixed formulation of quasi-reversibility to solve the Cauchy problem for Laplace’s equation”. In: *Inverse Problems* 21.3 (2005), pp. 1087–1104. DOI: 10.1088/0266-5611/21/3/018. URL: <https://hal-ensta-paris.archives-ouvertes.fr/hal-00876244>.
- [4] Jiang-Ren Chang, Weichung Yeih, and Min-Harn Shieh. “On the modified tikhonov’s regularization method for the cauchy problem of the laplace equation”. In: *Journal of Marine Science and Technology* 9 (Jan. 2001).
- [5] Xiaoliang Cheng et al. “A novel coupled complex boundary method for solving inverse source problems”. In: *Inverse Problems* 30 (Apr. 2014), p. 055002. DOI: 10.1088/0266-5611/30/5/055002.
- [6] Victor Isakov. “Uniqueness and Stability in the Cauchy Problem”. In: *Inverse Problems for Partial Differential Equations*. New York, NY: Springer New York, 1998, pp. 39–72. ISBN: 978-1-4899-0030-2. DOI: 10.1007/978-1-4899-0030-2_3. URL: https://doi.org/10.1007/978-1-4899-0030-2_3.
- [7] B. Jin and J. Zou. “Hierarchical Bayesian inference for Ill-posed problems via variational method”. In: *J. Comput. Phys.* 229 (2010), pp. 7317–7343.
- [8] Bangti Jin and Jun Zou. “Augmented Tikhonov regularization”. In: *Inverse Problems* 25.2 (Dec. 2008), p. 025001. DOI: 10.1088/0266-5611/25/2/025001. URL: <https://doi.org/10.1088/0266-5611/25/2/025001>.
- [9] Yixin Li and Xianliang Hu. *An artificial neural network approximation for Cauchy inverse problems*. 2020. arXiv: 2001.01413 [math.NA].
- [10] Martín Abadi et al. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. 2015. URL: <https://www.tensorflow.org/>.

- [11] Siddhartha Mishra. *A machine learning framework for data driven acceleration of computations of differential equations*. 2018. arXiv: 1807.09519 [math.NA].
- [12] M. Raissi, P. Perdikaris, and G. E. Karniadakis. “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations”. In: *Journal of Computational Physics* 378 (Feb. 2019), pp. 686–707. DOI: 10.1016/j.jcp.2018.10.045.
- [13] Weichung Yeh et al. “Solving The Inverse Cauchy Problem of The Laplace Equation Using The Method Of Fundamental Solutions and the Exponentially Convergent Scalar Homotopy Algorithm”. In: *Journal of Marine Science and Technology* 23 (Apr. 2015). DOI: 10.6119/JMST-014-0416-2.